TECHNICAL PORTFOLIO / 01 — ARCHITECTURE

AlphaChain ｜技術實作與系統設計

研究輔助系統的技術案例：如何把一則事件轉成**可驗證的資料、可追溯的研究流程，以及有邊界的輸出**。

Python　Pydantic v2　CSV　pytest　Git / GitHub

01 / 系統資料流

Event
事件輸入
›
Relationship
公司／供應鏈關係
›
Evidence
資料與引用來源
›
Validation
格式／時間／邊界
›
Output
FACT / INFERENCE / UNKNOWN

設計重點：流程不只追求「回答問題」，也要求能說明資料從哪裡來、何時可用，以及哪些結論因證據不足而必須停止。

02 / 代表性模組與用途

資料模型	`alphachain/models.py` 使用 Pydantic 定義與驗證研究資料結構，減少格式錯誤傳入後續步驟。
載入與查詢	`data_loader.py / relationship_engine.py` 載入 CSV 資料，查詢公司、關係與對應證據。
研究整合	`v1_end_to_end.py` 串接 V1 研究處理步驟，保留驗證與資料更新邊界。
結果呈現	`research_report.py` 將研究結果整理成可閱讀的輸出；不在呈現階段另外產生因果判斷。

03 / 設計取捨

POINT-IN-TIME / 時點一致性
研究特定時間點的事件時，只能使用當時可取得的資訊；對發布時間不明的證據保留限制，不用事後資料假裝事前已知。

HONEST STOP / 證據不足就停止
把 FACT（事實）、INFERENCE（推論）與 UNKNOWN（未知）分開，避免將缺少根據的影響路徑寫成確定結論。

技術聲明：這是 AI 輔助開發的個人研究原型，並非已上線的企業系統；本頁描述已接觸與參與驗證的模組，並不表示每個模組的程式碼都是本人獨立撰寫。

TECHNICAL PORTFOLIO / 02 — VERIFICATION

把「能執行」變成「能驗證」

專案開發除了功能完成，也重視測試結果、程式修改範圍、來源限制與是否足以支持研究結論。

04 / 一個實際遇到的邊界案例

案例：證據發布時間未知

1. **輸入資料**
   研究流程取得一筆真實來源資料，但發布時間欄位為 UNKNOWN。
2. **驗證判斷**
   無法證明資料在研究時點已公開，因此不應用它建立可靠的時點因果推論。
3. **系統輸出**
   保留未知並停止不受支持的因果傳遞；不為了產生結論而自動補齊缺失。
4. **我學到的事**
   資料有抓到不等於可以使用；驗證條件、時間與來源可信度會直接影響系統輸出的可靠性。

05 / 開發與驗證流程

VERSION CONTROL

版本管理與審查

透過 Git 分支與 GitHub Pull Request（程式修改審查請求）區隔變更，對照修改範圍、文件與測試結果，追蹤未解決的問題。

TESTING & DEBUGGING

測試與除錯

使用 pytest 的測試流程檢查資料驗證、研究輸出與邊界行為；依測試失敗或審查意見調整、再驗證。

06 / 人與 AI 的工作分工

本人主導	釐清問題、界定功能／非目標、評估研究方法、訂定驗證與停止條件、檢查成果是否符合專案方向。
AI 輔助	程式撰寫與重構、測試草案、除錯建議、技術文件整理及程式碼審查。
持續學習	閱讀程式與測試、理解資料流、累積可自己修改、除錯及維護的技術能力。

現階段的真實進度與能力邊界

V1 為研究原型，持續透過測試與研究評估驗證；V2 財務影響相關構想仍屬隔離實驗，未正式採用。沒有對外聲稱量產部署、財務預測準確率或獨立完成全部開發。

應徵方向：初階軟體工程師、資訊工程師、Python 應用開發。可於面試進一步說明專案設計、測試紀錄與個人的學習成果。